S.No: 1	Exp. Name: ***Registration Form using Table***	**Date: 2026-08-28**

Aim:
Build a registration form laid out inside an HTML table.

Form Fields

- **Name**: A text input with `name="name"`.
- **Password**: A password input with `name="password"`.
- **E-mail**: An email input with `name="email"`.
- **Phone Number**: A telephone input with `type="tel"` and `name="phone"`.
- **Sex**: Two radio buttons for "Male" and "Female", both with `name="sex"`.
- **Date of Birth**: Three dropdowns with `name="day"`, `name="month"`, and `name="year"`.
- **Languages Known**: Four checkboxes for English, Telugu, Hindi, and Tamil, each with a matching lowercase `name` attribute.
- **Address**: A textarea with `name="address"`.

Table Layout

- Place each label in one column and its input in the adjacent column.
- Set the table border to `1`.

Buttons

- A <button> element with `type="submit"` and the text "Submit".
- A <button> element with `type="reset"` and the text "Reset".

The form should look like this.

ID: A24126552277

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Registration Form

Name:	
Password:	
E-mail:	
Phone Number:	
Sex:	○ Male ○ Female
Date of Birth:	Day ▾ Month ▾ Year ▾
Languages Known:	☐ English ☐ Telugu ☐ Hindi ☐ Tamil
Address:	
Submit Reset	

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Registration Form</title>
</head>
<body>
<h2>Registration Form</h2>
<form>
        <table border="1">
                <tr>
                        <td>Name:</td>
                        <td>
                                <input type="text" name="name">
                        </td>
                        </tr>
                <tr>
                                <td>Password:</td>
                                <td>
                                        <input type="password"
name="password">
                        </td>
                </tr>
                <tr>

                                <tr>
                                        <td>E-mail:</td>
                                        <td><input type="email"
name="email">
                                        </td></tr>
                <tr>
                        <td>Phone Number:</td>
                        <td>
                                <input type="tel" name="phone">
                        </td>
                </tr>
                <tr><td>Sex</td><td><input type="radio" name="sex"
value="male">Male<br>
                        <input type="radio" name="sex" value="female">
Female</td>
                </tr>
                <tr>
```

```
                        <td>Date of Birth:</td>
                        <td><select name="day">
                                <option value=""> Day</option>
                                <option>1</option>
                                <option>2</option>
                                <option>3</option>
                                <option>4</option>
                                <option>5</option>
                                <option>6</option>
                                <option>7</option>
                        </select>
                                <select name="month">
                                        <option value="">Month</option>
                                        <option>january</option>
                                        <option>feb</option>
                                        <option>june</option>
                                </select>
                        <select name="year">
                                <option value="">Year</option>
                                <option>2005</option>
                                <option>2006</option>
                                <option>2007</option>
                        </select>
                        </td>
                </tr>
                <tr>
                        <td>Languages Known :</td>
                        <td>
                                <input type="checkbox" name="english"
value="english"> English<br>
                                <input type="checkbox" name="telugu"
value="telugu"> Telugu<br>
                                <input type="checkbox" name="hindi"
value="hindi"> Hindi<br>
                                <input type="checkbox" name="tamil"
value="tamil">
                                Tamil<br>
                        </td>
                </tr>
                <tr>
                        <td> Address:</td>
                        <td>
                                <textarea name ="address"> </textarea>
                        </td>
                </tr>
                <tr>
```

```
                        <td colspan="2">
                                <button type="submit"> Submit</button>
                                <button type="reset"> Reset </button>
                        </td>
                </tr>
        </table>
</form>




</body>
</html>
```

S.No: 2	Exp. Name: ***Ravi's Portfolio Page***	**Date: 2026-08-28**

Aim:

Ravi is designing his personal portfolio webpage. He wants each section to look visually distinct. Style the text in each section as described below.

- In the `<header>`, the `<h1>` should have `font-family: Verdana`, `color: royalblue`, `text-shadow: 3px 3px 8px gray`, and `text-align: center`.
- In the `<main>`, the `<h2>` should have `text-transform: uppercase`, `color: darkred`, and `letter-spacing: 2px`.
- In the `<main>`, the `<p>` should have `font-size: 16px`, `color: #333`, and `line-height: 1.7`.
- In the `<footer>`, the `<p>` should have `text-align: center`, `font-style: italic`, and `color: darkgreen`.

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Ravi’s Portfolio Page</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Portfolio</h1>
  </header>

  <main>
    <h2>about me</h2>
    <p>Hello! I'm Ravi, a passionate front-end developer who loves
designing simple and elegant web experiences for users.</p>
  </main>

  <footer>
    <p>© 2025 Ravi Sharma | Designed with creativity and code.</p>
  </footer>
</body>
</html>
```

styles.css

```
/*Write your code here*/
header h1{
        font-family:verdana;
        color:royalblue;
        text-shadow:3px 3px 8px gray;
        text-align: center;
}
main h2{
        text-transform:uppercase;
        color: darkred;
        letter-spacing:2px
}
main p{
        font-size:16px;
        color:#333;
        line-height: 1.7;
}
footer p{
        text-align:center;
        font-style:italic;
        color:darkgreen;
}
```

S.No: 3	Exp. Name: ***Simple Calculator***	**Date: 2026-09-09**

Aim:

You are provided with an HTML file that defines the structure of a basic calculator. The calculator supports the following arithmetic operations:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)

Your task is to complete the JavaScript file (script.js) so that the calculator functions correctly.

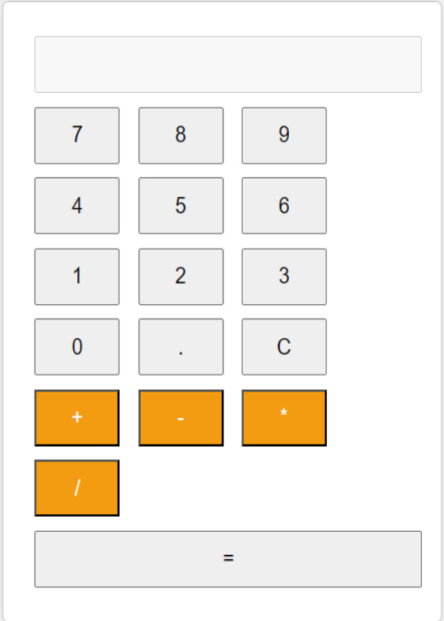
Provided Files

HTML File (index.html)

The HTML file already includes:

- A display area to show the current input or result.
- Buttons for digits (0–9).
- A decimal point button (.).
- Operation buttons (+, −, *, /).
- An equals button (=).
- A clear button (C).

You do not need to modify the HTML file.



Complete the JavaScript Functionality:

You must complete the logic in **script.js** by implementing the following functions correctly:

1. appendNumber(number)

This function is called when a number button is clicked.
It should:

- Allow multi-digit number input.
- Append the clicked digit to the current input.
- Handle decimal input properly (prevent multiple decimals in a single number).

2. setOperation(operation)

This function is called when an arithmetic operator (+, −, *, /) is clicked.
It should:

- Store the current input as the previous operand.
- Store the selected operation.
- Prepare the calculator to receive the next number.
- Prevent setting an operation if there is no current input.

3. calculate()

This function is called when the equals (=) button is clicked.
It should:

- Perform the selected arithmetic operation using the stored previous value and current value.
- Display the result.
- Handle division by zero by displaying:

`Cannot divide by zero`

If either the previous value or current value is empty, display:

`Error`

4. clearDisplay()

This function is called when the clear (C) button is clicked.
It should:

- Reset the current input.
- Reset the previous input.
- Reset the selected operation.
- Clear the display.

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Simple Calculator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            background-color: #f4f4f4;
        }
        .calculator {
            border: 1px solid #ccc;
            padding: 20px;
            border-radius: 5px;
            background-color: #fff;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        .calculator input, .calculator button {
            margin: 5px;
            padding: 10px;
            font-size: 16px;
        }
        .calculator input {
            width: 100%;
            box-sizing: border-box;
        }
        .calculator button {
            width: 22%;
        }
        .calculator button.operator {
            background-color: #f39c12;
            color: #fff;
        }
    </style>
</head>
<body>
    <div class="calculator">
        <input type="text" id="display" disabled>
```

```
        <br>
        <button onclick="appendNumber('7')">7</button>
        <button onclick="appendNumber('8')">8</button>
        <button onclick="appendNumber('9')">9</button>
        <br>
        <button onclick="appendNumber('4')">4</button>
        <button onclick="appendNumber('5')">5</button>
        <button onclick="appendNumber('6')">6</button>
        <br>
        <button onclick="appendNumber('1')">1</button>
        <button onclick="appendNumber('2')">2</button>
        <button onclick="appendNumber('3')">3</button>
        <br>
        <button onclick="appendNumber('0')">0</button>
        <button onclick="appendNumber('.')">.</button>
        <button onclick="clearDisplay()">C</button>
        <button class="operator" onclick="setOperation('+')">+</button>
        <button class="operator" onclick="setOperation('-')">-</button>
        <button class="operator" onclick="setOperation('*')">*</button>
        <button class="operator" onclick="setOperation('/')">/</button>
        <br>
        <button style="width: 100%;" onclick="calculate()">=</button>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

script.js

```
let currentInput = '';
let previousInput = '';
let operation = null;

function appendNumber(number) {
        if(number==="."&&currentInput.includes(".")){
                return;
        }
        currentInput+=number;
        updateDisplay();
}

function setOperation(op) {
    if(currentInput ===''){
                return;
        }
        previousInput = currentInput;
        currentInput="";
        operation=op;
        printToDisplay("");
}

function clearDisplay() {
        currentInput="";
        previousInput="";
        operation= null;
        printToDisplay("");

}

function calculate() {
        if(previousInput ==="" || currentInput ===""){
                printToDisplay("Error");
                return;
        }
        let prev=parseFloat(previousInput);
        let current=parseFloat(currentInput);
        let result;
        switch(operation){
                case "+":
                        result=prev+current;
                        break;
                case"-":
                        result=prev-current;
```

```
                        break;
                case "*":
                        result=prev*current;
                        break;
                case "/":
                        if(current === 0){
                                printToDisplay("Cannot divide by
zero");
                                return;
                        }
                        result=prev/current;
                        break;
                default:
                        printToDisplay("Error");
                        return;
        }
        currentInput=result.toString();
        previousInput="";
        operation=null;
        updateDisplay();

}

function updateDisplay() {
    document.getElementById('display').value = currentInput;
}

function printToDisplay(message) {
    document.getElementById('display').value = message;
}
```

S.No: 4	Exp. Name: ***Dynamic To-Do List with Modals***	**Date: 2026-09-30**

Aim:

You have joined a front-end team building a productivity app. The HTML and CSS for a To-Do List app are already provided. Write the JavaScript in `script.js` to make the app interactive.

Your Task

1. Add Task

When the user types a task and clicks the `Add` button, add a new `<li>` to the list with `id="task-list"`. Each item must include:

- A `<span>` holding the task text
- Two buttons labelled `Edit` and `Delete`

2. Edit Task with Modal

Clicking `Edit` on a task should:

- Open `#edit-modal` by setting its `display` to `block`
- Show the current task text inside `#edit-input`
- Update the task text when the user clicks `Save`

The modal should close when:

- The user clicks `Save`
- The user clicks the close icon at the top right
- The user clicks outside the modal

3. Delete Task with Confirmation

Clicking `Delete` on a task should:

- Open `#delete-modal` by setting its `display` to `block`
- Remove the task if the user clicks `Yes`
- Close the modal without deleting if the user clicks `No`
- Close the modal if the user clicks outside the confirmation box

Note:

- Trigger the **Add** action when the user clicks the **Add** button. You may implement this using either the button's *click* event or the form's submit event (using `event.preventDefault()` if needed).

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Dynamic To-Do List</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <form id="task-form">
      <input type="text" id="task-input" placeholder="Add a new task"
required />
      <button type="submit" id="add-task">Add</button>
    </form>
    <ul id="task-list"></ul>
  </div>

  <!-- Edit Modal -->
  <div id="edit-modal" class="modal">
    <div class="modal-content">
      <span class="close-btn" id="close-edit-modal">&times;</span>
      <h2>Edit Task</h2>
      <input type="text" id="edit-input" />
      <button id="save-edit">Save</button>
    </div>
  </div>

  <!-- Delete Confirmation Modal -->
  <div id="delete-modal" class="modal">
    <div class="modal-content">
      <p>Are you sure you want to delete this task?</p>
      <button id="confirm-delete">Yes</button>
      <button id="cancel-delete">No</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
const taskForm=document.getElementById("task-form");
const taskInput=document.getElementById("task-input");
const tasklist=document.getElementById("task-list");

const editModal=document.getElementById("edit-modal");
const editInput=document.getElementById("edit-input");
const saveEdit=document.getElementById("save-edit");
const closeEditModal=document.getElementById("close-edit-modal");

const deleteModal=document.getElementById("delete-modal");
const confirmDelete= document.getElementById("confirm-delete");
const cancelDelete=document.getElementById("cancel-delete");
let currentTask=null;
taskForm.addEventListener("submit",function(event){
        event.preventDefault();
        const taskText=taskInput.value.trim();
        if(taskText===""){
                return;
        }
        const li=document.createElement("li");
        const span=document.createElement("span");
        span.textContent = taskText;
        const editButton=document.createElement("button");
        editButton.textContent= "Edit";
        const deleteButton = document.createElement("button");
        deleteButton.textContent= "Delete";
        li.appendChild(span);
        li.appendChild(editButton);
        li.appendChild(deleteButton);
        tasklist.appendChild(li);
        taskInput.value="";
        });
        tasklist.addEventListener("click",function(event){
                if(event.target.textContent === "Edit"){
                        currentTask=event.target.parentElement;

editInput.value=currentTask.querySelector("span").textContent;
                        editModal.style.display="block";
                }
                if(event.target.textContent === "Delete"){
                        currentTask=event.target.parentElement;
                        deleteModal.style.display = "block";
                }
        });
```

```
saveEdit.addEventListener("click",function(){
        const newText=editInput.value.trim();
        if(newText !== "" && currentTask){
                currentTask.querySelector("span").textContent= newText;
                editModal.style.display="none";
                currentTask=null;
        }
});
closeEditModal.addEventListener("click",function(){
        editModal.style.display = "none";
        currentTask=null;
});
confirmDelete.addEventListener("click",function() {
        if(currentTask){
                currentTask.remove();
        }
        deleteModal.style.display="none";
        currentTask=null;
});
cancelDelete.addEventListener("click",function(){
        deleteModal.style.display="none";
        currentTask=null;
});
window.addEventListener("click",function(event){
        if(event.target === editModal){
                editModal.style.display="none";
                currentTask=null;
        }
        if(event.target === deleteModal){
                deleteModal.style.display="none";
                currentTask=null;
        }
});
```

S.No: 7	Exp. Name: ***Creating and Nesting Function and Class Components***	**Date: 2026-10-22**

Aim:

You are developing a React application for a digital library. The application should display information about a book by creating and nesting React components. In this activity, the React application setup has already been provided. Your task is to create the **Book** function component and the **App** class component and nest the **Book** component inside the **App** component.

Your Task

1. Inside **Book.jsx**, create a function component named **Book** that returns a `<div>` with the class **book-card**, containing:
    - An `<h2>` element with the text "The Great Gatsby".
    - A paragraph element with the text "Author: F. Scott Fitzgerald".

Export the **Book** component.

2. In the **App.jsx** file:
    - Create a class component named **App** that extends **Component**.
    - Import the `Book` component from `./Book`.
    - Inside the `render()` method, display an `<h3>` element with the text "Books".
    - Create a `<div>` with the class name **library**, and nest the `<Book />` component inside it.
    - Return the JSX from the `render()` method.
    - Export the **App** component as the default export.

Expected Output:

When the application is executed, it should display:

```
Books
--------------------------------
The Great Gatsby
Author: F. Scott Fitzgerald
--------------------------------
```

where the **Book** component is nested inside the **App** component.

Note:

- Both **App.jsx** and **Book.jsx** are in the same path.
- Initially, the page will show a placeholder until the required code is completed.

Source Code:

functionAndClassComponents/src/App.jsx

```
import { Component } from "react";
import Book from "./Book";
import ErrorBoundary from "./components/ErrorBoundary";

class App extends Component {
        render() {
                return (
                        <div>
                                <h3>Books</h3>
                                <div className="library">
                                        <Book />
                                </div>
                        </div>
                );
        }
}
export default App;
```

functionAndClassComponents/src/Book.jsx

```
// Create the Book function component here...
function Book(){
        return(
                <div className="book-card">
                        <h2>The Great Gatsby</h2>
                        <p>Author: F. Scott Fitzgerald</p>
                </div>
        );
}
export default Book;
```